

2026 MITRE eCTF

University of Denver | Final Design Documentation

Advisors: Dr. Nathan Evans (nathan.evans@du.edu), Dr. Rinku Dewri (rinku.dewri@du.edu)

Members: Jerome Althoff (jerome.althoff@du.edu), Jacob Fishbein (jacob.fishbein@du.edu), Domonic Velarde (domonic.velarde@du.edu), Warren Kankwe (warren.kankwe@du.edu), LaRon Latin (laron.latin@du.edu), Jason Velvick (jason.velvick@du.edu), Sy Pretto (lilia.pretto@du.edu), Adrian Rodriguez (adrian.rodriguez@du.edu)

Introduction: Secure Transfer of Circuit Fabrication Data

We are a team of graduate students from the University of Denver working toward the design and implementation of a secure file transfer firmware for multiple Hardware Security Modules (HSMs). The files are supposed to be locally accessible under a PIN and a permission model via a management interface, and transferable between two HSMs subject to authentication and authorization constraints. This design uses an MPU-enforced split between unprivileged command handling and privileged security services, with sensitive material and protected storage accessible only through a constrained interface. Files are encrypted at rest with AES-GCM under per-file keys derived from a device-local secret, while inter-device metadata and transfer control are protected using a deployment-shared secret, HMAC-SHA256, and nonce-derived session keys. The design also includes explicit software countermeasures against glitch and fault-injection attacks.

Table of Contents

Section 1 – What We Are Protecting Against:	3
Requirements Mapping.....	3
Initial Concepts.....	3
Figure 1 – Normal Operations	3
Final Design	4
Section 2 – How The System Works.....	5
Figure 2 – Basic Global Concept	5

Trust Boundaries	5
Figure 3 – Visual of Privileged vs. Unprivileged Separation	6
Separation of Duties: Kernel Code and User Code Segregation	6
Secure Local Storage	6
PIN Security	7
Capability System	7
Figure 4 – Capability Requirements Visual	7
Local Board Security	7
Random Number Generation	7
Transmission Security (HSM to HSM)	7
Specifics on Implementation	8
Section 3 – Cryptography Overview	9
Main Cryptographic Method	9
Figure 5 – Interrogate Command Flow	10
Figure 6 – Receive Command Flow	11
Keys and Key Derivation Function (KDF)	11
Figure 7 – Key Derivation	11
Function	Error! Bookmark not defined.
Figure 8 – Nonce for File Key Derivation	12
Figure 9 – Nonce for Transfer	12
Section 4 – Meeting the Security Requirements	13

Section 1 – What We Are Protecting Against

Requirements Mapping

This section outlines how our design satisfies the MITRE eCTF functional and security requirements through internally defined granular security requirements (GSRs). Each GSR maps MITRE requirements and enforced through specific system design components.

Granular Security Requirements (GSRs):

- GSR1: The system shall prevent brute-force PIN attacks, even under repeated reset conditions.
- GSR2: The system shall ensure that only authenticated and authorized HSMs can perform file operations.
- GSR3: The system shall ensure confidentiality and integrity of all data at rest and in transit.
- GSR4: The system shall prevent unauthorized data injection and replay attacks during HSM-to-HSM communication.
- GSR5: The system shall prevent unauthorized execution of privileged operations from untrusted code.
- GSR6: The system shall minimize the impact of physical and side-channel attacks through secure coding and hardware-supported protection.

Mapping to MITRE Security Requirements:

- SR1 → GSR2
- SR2 → GSR1
- SR3 → GSR2, GSR3, GSR4

Initial Concepts

The embedded system involves two separate HSM devices (Texas Instruments MSP-LITO-L2228). In normal operations, data is transferred from the customer site to the factory via couriered HSMs.

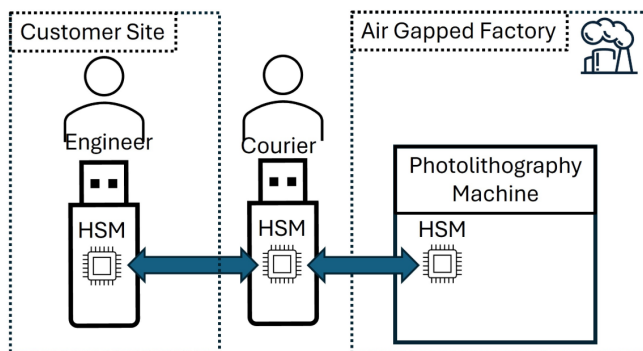


Figure 1 – Normal Operations

There exists a potential for a malicious actor to directly attack the HSM or to attempt to attack the transfer of data. The main methods we will use to address this vulnerability are the use of cryptographic algorithms and secure secret storage (detailed in section 3) to ensure data in storage and transit remains confidential and integrity protected. Some threats which we initially considered: (1) brute forcing the PIN, (2) bypassing security configuration, (3) side channel/physical board attacks, (4) arbitrary code execution, (5) data interception during transfer (6) data injection. Considering these in ascending order of severity:

(1) An attacker can try random guesses of the PIN in an attempt to gain full control of a HSM. The total space of PINs is 16^6 which can be exhausted in a small amount of time.

(2) An attacker who can modify memory or flash locations will be able to downgrade the security configuration. Without modifying the security logic, an attacker who reads the documentation and understands the data structure will be able to add arbitrary permissions to interact with the filesystem and the neighboring HSM's filesystem (read only) or change secrets (keys and PIN).

(3) If it is possible to infer AES keys via passive monitoring of power consumption or RF emissions, or via other side channels/physical means, attackers will also gain full read of both local and neighboring HSMs, as well as write for the local HSM.

(4) If an attacker can exploit software vulnerabilities to gain arbitrary code execution, they may be able to bypass security checks entirely. Even if we are using secure hardware elements, any software-imposed limits can be easily bypassed, as well as any other cryptographic operations.

(5) (6) Finally, data flowing between two HSMs can be intercepted during transfer, revealing the contents of transferred files. Similarly, modified data may be injected into the transmit channel to force a HSM to receive malformed files.

Final Design

After iterating through design and testing, we finalized on implementing the following security requirements based on the anticipated threat scenarios:

- The system shall prevent brute-force PIN attacks, even under repeated reset conditions.
- The system shall prevent unauthorized modification of permissions, keys, and PIN data.
- The system shall protect against unauthorized data interception and ensure confidentiality of all transmitted data.

- The system shall protect against unauthorized data injection or replay during HSM-to-HSM communication.
- The system shall prevent leakage of cryptographic keys through side-channel attacks.
- The system shall protect against arbitrary code execution that could bypass security enforcement mechanisms.

Section 2 – How The System Works

The critical path of the system consists of two HSMs needing to securely transfer sensitive data to and from one another. Specifically, there are six explicit functional requirements for the HSMs, three explicit security requirements, as well as best practice standards for separation of duties, secure local storage, local (board centric) security, and transmission security.

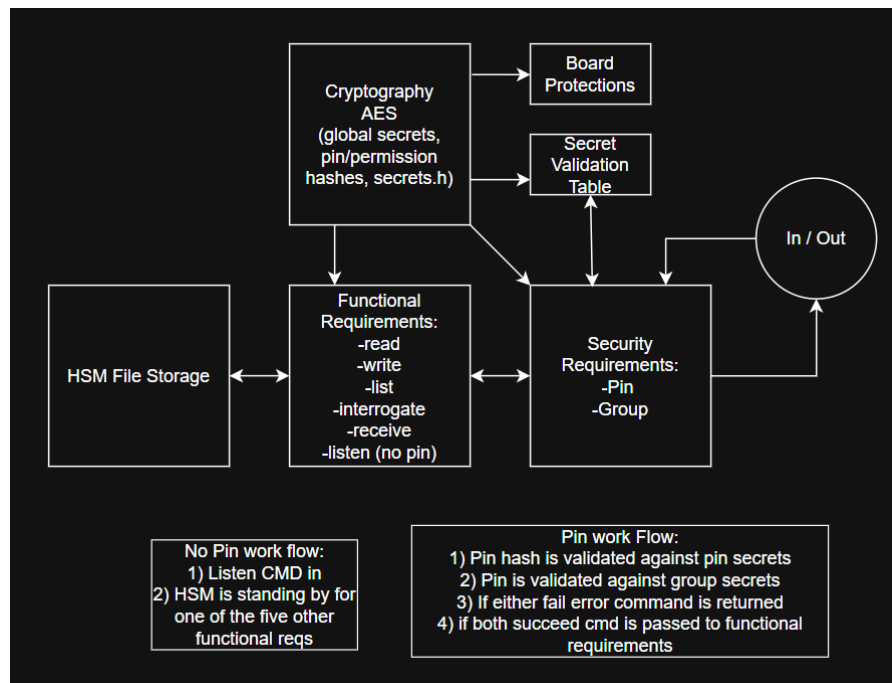


Figure 2 – Basic Global Concept

Trust Boundaries

Figure 3 illustrates the system architecture and highlights trust boundaries of the system in code. The system enforces multiple trust boundaries:

1. **User/Kernel Boundary:** User code operates without access to sensitive resources. All cryptographic operations and key handling operations are executed within kernel space.
2. **HSM-to-HSM Boundary:** All communication between HSMs occurs over UART and is secured using authenticated encryption.

3. Storage Boundary: All data stored in flash memory is encrypted and integrity-protected.

These boundaries ensure that compromise in one domain does not lead to full system compromise.

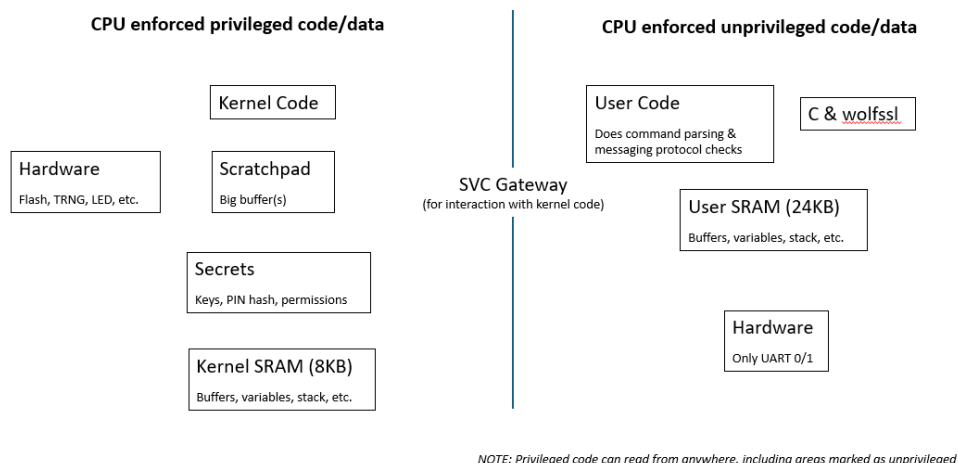


Figure 3 – Visual of Privileged vs. Unprivileged Separation

Separation of Duties: Kernel Code and User Code Segregation

We use separation of duties to ensure confidentiality and integrity of the files and HSM functionality. The system enforces separation between privileged (Kernel) code and unprivileged (User) code using the Memory Protection Unit (MPU) of the Arm Cortex M0+ processor. Kernel code executes in a protected memory region with exclusive access to cryptographic operations, key material, and security-critical logic. User code operates in an unprivileged region and is limited to parsing commands and interacting with the UART interfaces. All sensitive operations must transition into kernel space through controlled and validated SVC interfaces.

By placing the cryptography logic within the Kernel code, we ensure that the methods as well as security assets (PIN(s), keys, permissions structure, etc.) and the ciphertexts are secure during operations. This separation provides the privileged and unprivileged spaces used to comply with the functional and security requirements.

Secure Local Storage

The security configuration is stored in a read-only section of flash memory. This memory region is configured as read-only with limited logical access, restricted to the security validation portion of the code, and established during MPU setup. The security configuration consists of cryptographic key(s), group permission structure, and PIN hash. Files are stored locally in encrypted form using keys derived from device-specific secrets.

PIN Security

PIN information is stored in salted, SHA-256 hashed-form in the secure local storage. We incorporated a four second delay in the event an invalid PIN is presented. This is designed as a mitigation to brute force attacks. We also delay for up to one second during the boot process to prevent PIN brute forcing with board resets. Assuming one second per PIN attempt, with 16^6 possible PIN values, on average an attacker is expected to take 97 days to correctly guess a PIN. With these mitigations, we aim to prevent an attacker from performing a brute force attack to determine the secret PIN in a reasonable amount of time, even with system resets.

Capability System

For each command executed in the HSMs, the system receives a capability after PIN verification. Later, during the operation, we check that the system has the required capability to run a specific command, along with group permissions. This ties the PIN verification step to the steps undertaken to perform the actual operation. The following figure illustrates the logic.

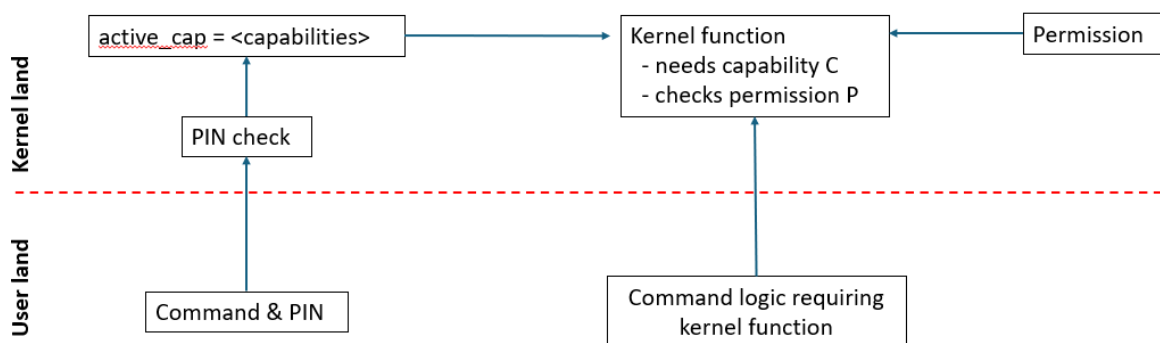


Figure 4 – Capability Requirements Visual

Local Board Security

The system minimizes the attack surface by restricting access to only the hardware modules and interfaces required for essential functionality. Unused peripherals and interfaces are not initialized by the firmware, reducing exposure to unintended interactions. We do use the onboard TRNG for random number generation.

Random Number Generation

We use the onboard TRNG for random number generation. The output is XOR-mixed with a HMAC-SHA256 software-based random number generator.

Transmission Security (HSM to HSM)

During transfer, files remain encrypted end-to-end. A temporary transfer key is derived using shared secrets and random nonces generated by both HSMs. The sending HSM

provides key and file metadata in encrypted form to the receiving HSM, which uses it to remove existing encryption of the file data and re-encrypt it to local ciphertext before storage. Details of the cryptograph methods are in Section 3.

Specifics on Implementation

The core implementation of the MPU-protected code paths are in **security.c** and the modified file metadata structure in **filesystem.h**. Files are stored with the following structure:

```
typedef struct {
    uint32_t in_use; // FILE_IN_USE if in use
    group_id_t group_id;
    char name[MAX_NAME_SIZE];
    uint16_t contents_len;
    uint8_t aes_gcm_iv[MAX_AES_IV_SIZE];
    uint8_t aes_gcm_tag[MAX_AES_TAG_SIZE];
    uint8_t contents[MAX_CONTENTS_SIZE];
} file_t;
```

We also define **file_header_t**, which is a duplicate, except that it excludes the contents. This is for simplifying the calculation of the number of bytes at various points in the code.

The relevant macros and functions in **security.c** are as follows:

- **EQ_CHECK_BARRIER** is a macro used when equality comparisons (of non-zero values) are needed in security-sensitive operations. This barrier does a normal comparison of actual versus expected values, then compares against all zeroes, then all ones, then the result of XOR's equality to zero. Lastly, it does one final check of the negatively initialized (with a checkerboard pattern) state variable in case the earlier statements were skipped. If any of these fail, the macro returns the supplied error code. Any of these return statements cause the calling function to fail. Otherwise, the equality check succeeds and execution continues.
- **SECURE_CAP_CHECK**, which calls **EQ_CHECK_BARRIER**, is a macro used to check if the system has a specific capability. It immediately copies the active capability and clears it. This active capability value is set after successful PIN check. We use full byte values with maximal Hamming distance for capabilities to reduce the chance of glitching attacks.
- **SECURE_PERM_CHECK** is a macro used to check if a group has a required permission. It loops the supplied permission structure (which could be from local or

remote) and uses **EQ_CHECK_BARRIER** and data-oriented computations (not branching) to detect the match or non-match.

- **join_bytes** has the simple job of serializing bytes from multiple sources into one byte stream. Its arguments are pointers paired with the number of bytes to copy from each. A `NULL` param value indicates the end of the list of arguments. This function is used for key derivation, formatting file headers, and other use cases.

- **copy_with_transform** takes an input buffer and applies an encryption or decryption step, setting aside the result in an output buffer. The destination can be SRAM or flash. We have reserved structures in a region of kernel flash memory, called the “scratchpad.” These intermediate locations limit the exposure of plaintext data. The function operates in chunks equal to the flash memory page size, both to optimize the flash write use cases, and to allow for cryptographic operations on data which would otherwise exceed available SRAM. The function has the ability to skip a specified number of bytes in order to preserve the file header information as plaintext. This information will need to be read before decryption and integrity validation can take place so it cannot be encrypted, hence the need for this offset.

- **create_key** uses a supplied secret to compute Hash-based Message Authentication Code (HMAC) on the supplied “nonce” byte stream. For local storage, the nonce bytes are and must be deterministic, so they include file information and a context label. For transfer, random bytes are used. This process adds a level of indirection and mitigates the compromise of an individual file key or transfer key.

Section 3 – Cryptography Overview

The implementation uses symmetric encryption for simplicity's sake and performance. HSMs are provisioned with a shared key, used for transfer of communications, as well as unique local keys, used for file storage. The system will allow the sending board to implicitly verify the receiving board for any data transfer over UART. A third local secret is used in the random number generator.

Main Cryptographic Method

We use AES GCM for symmetric cryptographic operations with authentication. This gives the system confidentiality and integrity protection, both for data in transit and data at rest. Malicious changes of any of the protected fields are detected by WolfSSL during decryption. IV's are randomly generated and stored with file headers or serialized during data transfer. So is the tag, which is the MAC for the ciphertext. The following describes the

protocols that we use. Cryptographically relevant data flow for the interrogate and receive commands are pictured in Figures 5 and 6.

Local storage derives a key using the file header (excluding IV and tag) concatenated with the file UUID and a fixed context label. The secret passed to key derivation is the local key for the HSM. This ensures that if a file is somehow flashed to a different HSM with a different local key, decryption will fail. Read and write from the same HSM will always derive the same key for the same file.

Transfer uses random bytes (nonce) from both HSMs during key derivation. The listener HSM derives a key for transfer by combining the requestor's nonce with its own nonce, and by using the shared key. The inclusion of random nonces from both communicating HSMs ensures that each transfer session is unique, preventing replay attacks. Any attempt to reuse previously captured messages will fail due to mismatched key derivation inputs. This transfer key is used by the listener HSM to encrypt its own local decryption key (which is derived as described in the previous paragraph,) the UUID, and the file header together as a byte stream. Then the local ciphertext is copied from local storage as-is and appended to the previously encrypted payload. The receiver HSM derives the same transfer key using the nonces, decrypts the remote local key, and then decrypts the received ciphertext. Then, the receiver HSM derives its own local key, re-encrypts the file and stores the file the same way it would for the write command. This key encapsulation approach meets timing requirements without impacting security requirements.

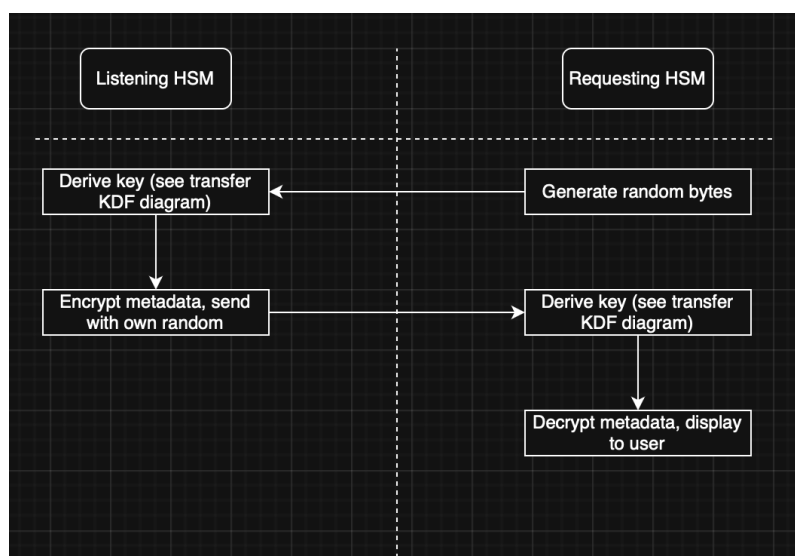


Figure 5 – Interrogate Command Flow

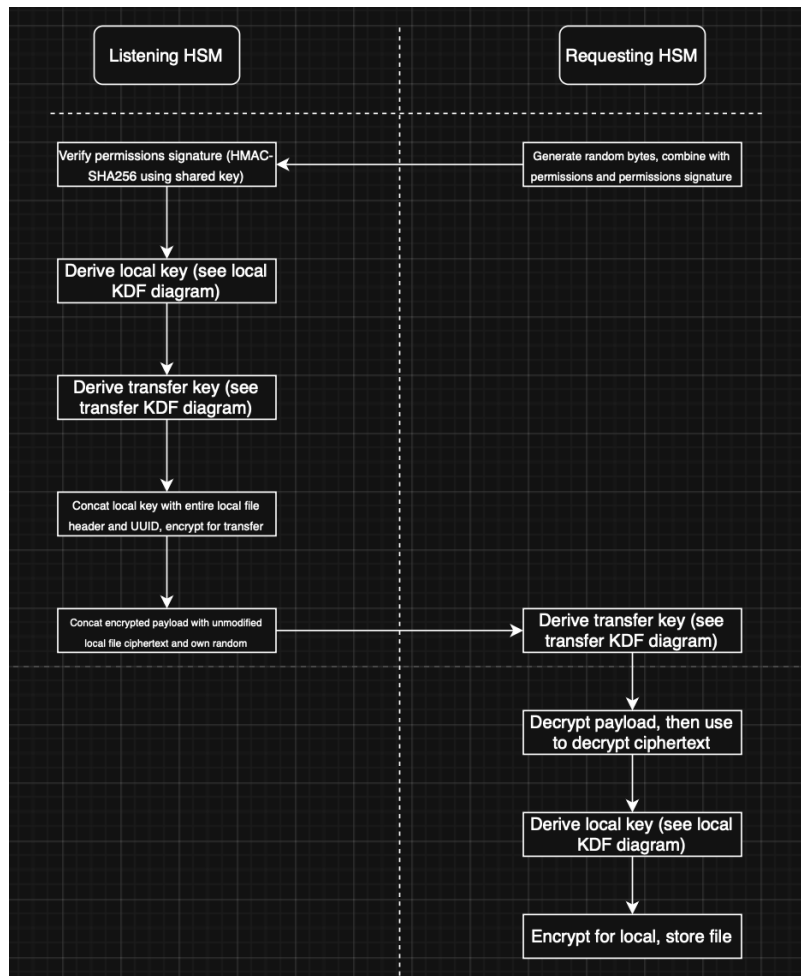


Figure 6 – Receive Command Flow

The system uses 128 bit keys for all operations. File and transfer keys are not stored after being generated. After keys are derived, they are used in operations, and immediately discarded. The key derivation process is pictured below.

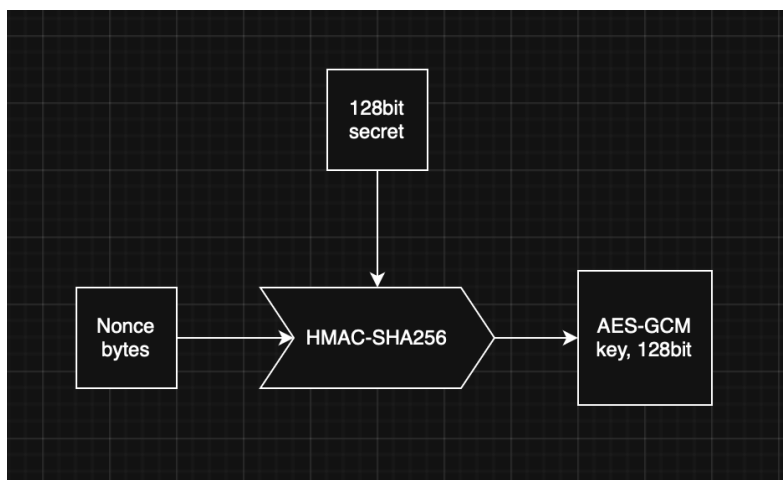


Figure 7 – Key Derivation Function

Local storage uses `aes_128_local_key` from `secrets.h` as the 128-bit secret input to the KDF. Transfer communication uses `aes_128_shared_key` from `secrets.h` as the 128-bit secret input to the KDF. Nonce byte generation for local storage is pictured in Figure 8. Nonce byte generation for transfer communication is pictured in Figure 9 (note, transfer of metadata for the interrogate command swaps the order of the local random and remote random bytes; otherwise, it is identical.)

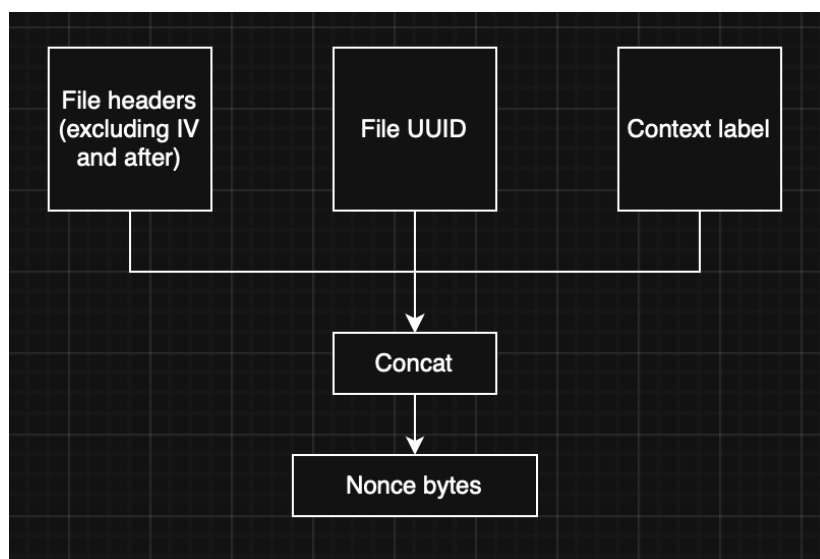


Figure 8 – Nonce for File Key Derivation

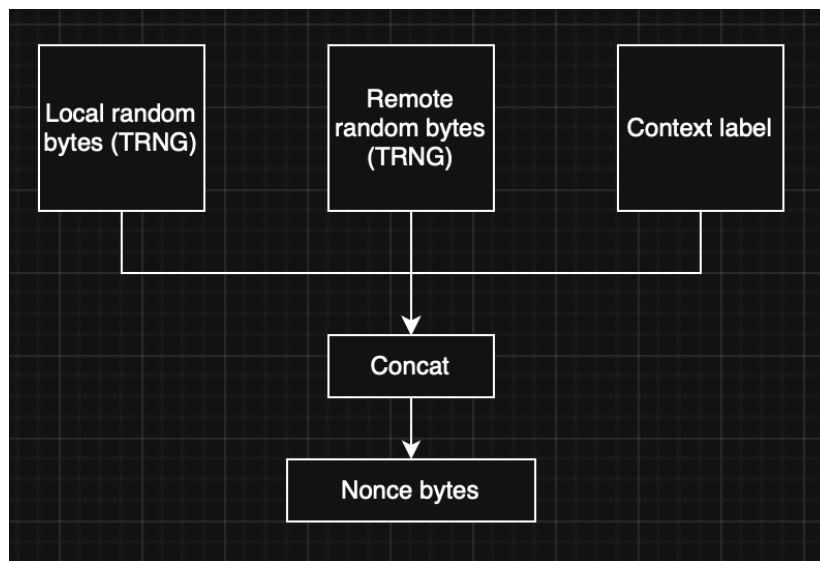


Figure 9 – Nonce for Transfer

Section 4 – Meeting the Security Requirements

We meet security requirements through the GSRs which are then enforced throughout the implementation.

- SR1 → GSR2: Only authenticated and authorized HSMs may perform file operations.
 - Implementation: Mutual authentication using shared key and AES-based handshake; hardened PIN check and capability architecture; hardened permission validation.
- SR2 → GSR1: PIN-protected actions require correct PIN knowledge.
 - Implementation: Secure PIN hashing and validation with enforced delay and boot-time penalty to mitigate brute-force attempts.
- SR3 → GSR2, GSR3, GSR4: Only valid HSMs may send/receive files, and data is protected.
 - Implementation: AES-GCM transfer encryption ensures confidentiality and integrity; transfer key derivation uses shared secrets and nonces; replay protection enforced via nonce uniqueness.